

Le plan de la formation

❖ Introduction générale

- Qu'est ce que le JavaScript
- L'historique du JavaScript

❖ Les bases du JavaScript

- Insertion du code JavaScript dans une page html
- Les variables
- Les opérateurs
- Les structures de contrôle
- Les fonctions
- Lire/Ecrire

❖ Les événements



Le plan de la formation

❖ Introduction générale

- Qu'est ce que le JavaScript
- L'historique du JavaScript

❖ Les bases du JavaScript

- Insertion du code JavaScript dans une page html
- Les variables
- Les opérateurs
- Les structures de contrôle
- Les fonctions
- Lire/Ecrire

❖ Les événements



Introduction générale

Qu'est ce que le JavaScript?

Le JavaScript est un langage de programmation de scripts orienté objet.

Langage qui permet aux développeurs
d'écrire du code source qui sera analysé
Par l'ordinateur.

Langage interprété

Utilise les objets pour communiquer
avec le monde extérieur

Le JavaScript s'inclut directement dans la page web et permet de dynamiser une page html en ajoutant des interactions avec l'utilisateur, des animations et de l'aide à la navigation.

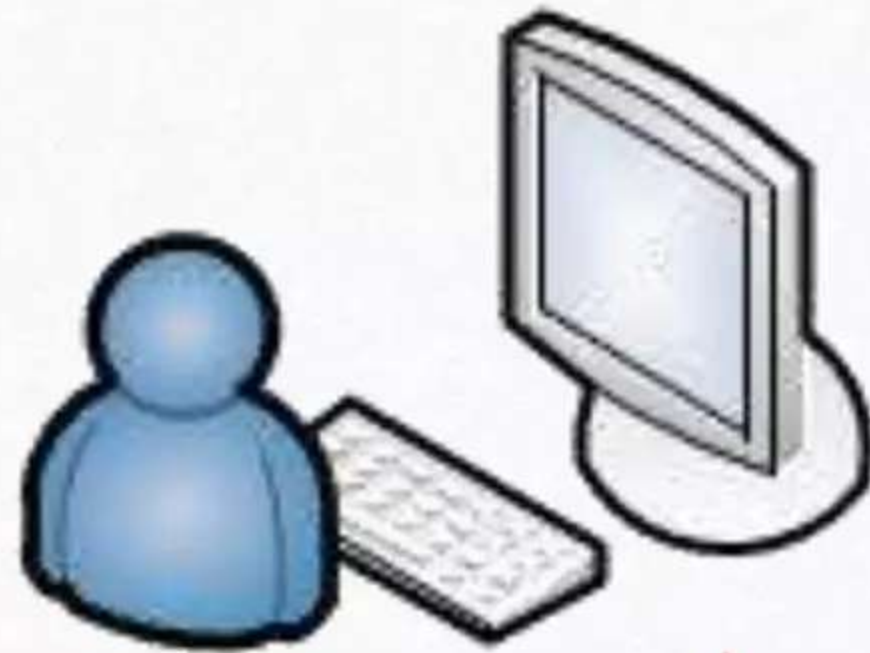
Exemples:

- ✓ Afficher/masquer du texte
- ✓ Faire défiler des images
- ✓ Créer un diaporama avec un aperçu « grand » des images
- ✓ Créer des infobulles



Le JavaScript est un langage dit **Client-Side** c.-à-d. que les scripts sont exécutés par le navigateur chez le client contrairement aux langages **Server-Side** qui sont exécutés par le serveur web (exemple: **PHP**).

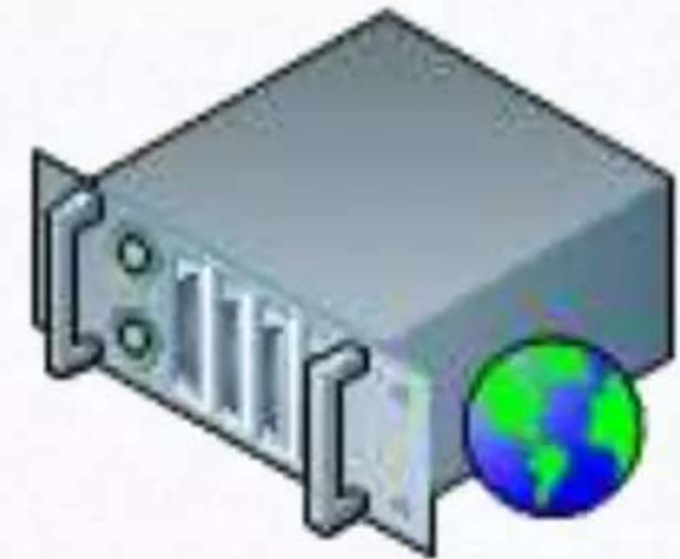
Visiteur Client



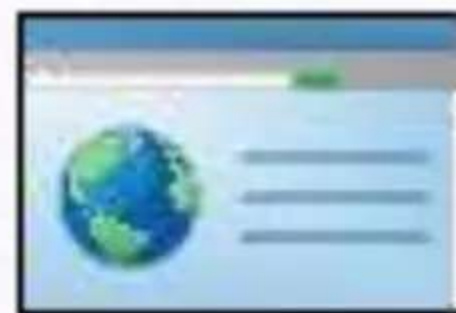
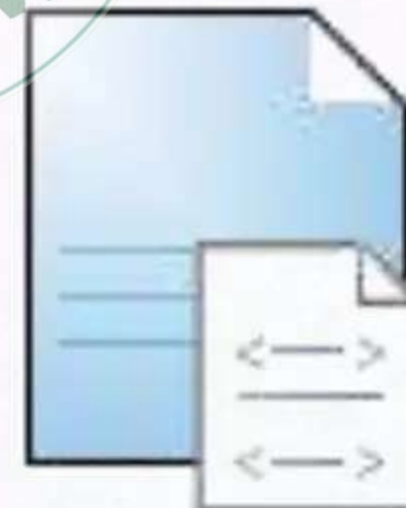
Demande d'une page



Serveur



Le serveur
envoie la page
HTML et le
Javascript



Le navigateur affiche le HTML
et exécute le Javascript



Le JavaScript est un langage basé événement (**event-driven**):

- ✓ Il permet de manipuler les événements de la souris, les menus déroulants, les messages d'alerte, les fenêtres, les cadres, les données des formulaires, et leur associer des actions ou des fonctions.
- ✓ Il peut être utilisé pour vérifier la validité des données fournies par l'internautes.



L'historique du JavaScript

JavaScript a été initialement développé par Netscape en 1995 puis standardisé par l'ECMA international (European Computer Manufacturers Associations) sous le nom d'ECMAScript.



Brendan Eich l'inventeur du JavaScript

Le plan de la formation

❖ Introduction générale

- Qu'est ce que le JavaScript
- L'historique du JavaScript

❖ Les bases du JavaScript

- Insertion du code JavaScript dans une page html
- Les variables
- Les opérateurs
- Les structures de contrôle
- Les fonctions
- Lire/Ecrire

❖ Les événements



Les base du JavaScript



Le plan de la formation

- ❖ Introduction générale
 - Qu'est ce que le JavaScript
 - L'historique du JavaScript

- ❖ Les bases du JavaScript

- **Insertion du code JavaScript dans une page html**
- Les variables
- Les opérateurs
- Les structures de contrôle
- Les fonctions
- Lire/Ecrire

- ❖ Les événements



Insertion du code JavaScript dans une page html

L'intégration du code JavaScript dans une page html peut se faire de 3 manières:

- En utilisant la balise `<SCRIPT>`
- En mettant le code dans un fichier externe
- En mettant en place un gestionnaire d'événements

En utilisant la balise <SCRIPT>

Le code JavaScript peut être inséré dans n'importe quel endroit de la page web.

Il est généralement placé dans la balise d'entête <HEAD> et </HEAD>. Cependant les événement JavaScript sont insérés dans le corps de la page web entre la balise <BODY> et </BODY> en tant qu'attribut d'un marqueur html.

Syntaxe:

```
<SCRIPT langage="JavaScript" / type="texte/JavaScript">  
<!--  
Le code du script  
-->  
</SCRIPT>
```

```
1  <!DOCTYPE html>
2  <html>
3
4  <head>
5      <title> Interaction!! </title>
6  </head>
7
8  <body>
9
10 <script type="text/javascript">
11
12 alert('hello guys !|');
13
14 </script>
15
16 </body>
17 </html>
```



Insertion dans un fichier externe

Le code JavaScript est placé dans un fichier indépendant sauvegardé avec l'extension **.js**
l'inclusion du fichier externe dans le code html via la ligne de code suivante:

```
<SCRIPT langage="JavaScript" src="url/fichier.js"> </SCRIPT>
```

Mise en place d'un gestionnaire d'événements

Dans le navigateur certaines actions effectuées par l'internaute donnent lieu à des événements (exemple: entrée de souris, clic sur bouton par une souris,..)



Un gestionnaire d'événements mise en place sera automatiquement exécuté lorsque l'événement correspondant se présentera.

Syntaxe pour définir un gestionnaire d'événements:

<balise **nom événement** ="code JavaScript à exécuter">

```
1  <!DOCTYPE html>
2  <html>
3
4  <head>
5      <title> Interaction!! </title>
6  </head>
7
8  <body>
9
10 <span onclick="alert('cliquez moi!');"> cliquez ici </span>
11
12 </body>
13 </html>
```



Le plan de la formation

❖ Introduction générale

- Qu'est ce que le JavaScript
- L'historique du JavaScript

❖ Les bases du JavaScript

- Insertion du code JavaScript dans une page html
- **Les variables**
- Les opérateurs
- Les structures de contrôle
- Les fonctions
- Lire/Ecrire

❖ Les événements



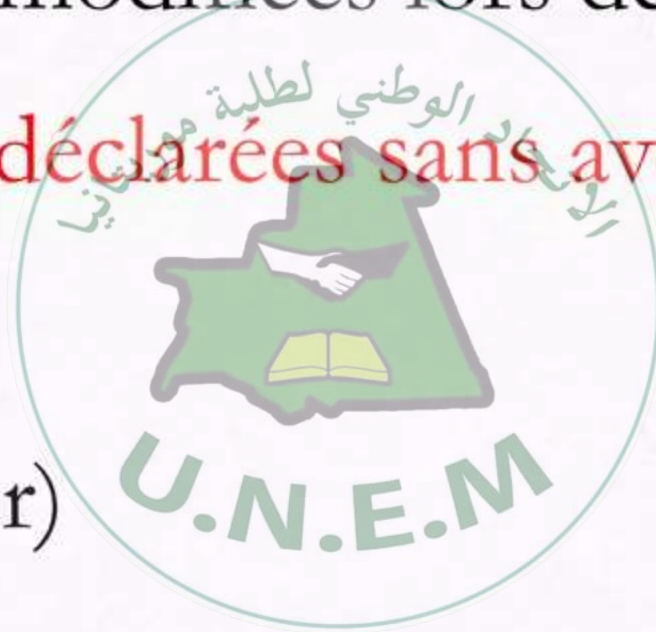
Les variables

- Une variable est un espace de stockage sur votre ordinateur identifié par un nom permettant d'enregistrer tout types de données (valeur numérique, chaine de caractères ...) qui pourront être modifiées lors de l'exécution du programme.
- En JavaScript, les variables sont déclarées sans avoir besoin de préciser leur type.

Type de données:

- Les nombres (NAN : not a number)
- Les chaines de caractères

pour utiliser les caractères spéciaux dans les chaines de caractères, il faut les précédé d'un antislash(\) \\ , \', \", \n ...



-Les booléens(true , false)

-Les variables de type NULL: un mot caractéristique qui signifie qu'une variable ne contient pas de donnée.

Déclaration des variables:

En JavaScript, nos variables sont typées dynamiquement.

Var myVariable;

JavaScript est un langage sensible à la casse!

myVariable = 5; //affecter une valeur à une variable.

Var myVariable1, myVariable2 = 4 , myVariable3;

Var myVariable1, myVariable2 ;

myVariable1 = myVariable2 = 2;

Pour tester l'existence d'une variable et/ou vérifier son type, on utilise l'instruction **typeof**

```
Var value = 2;
```

```
alert(typeof value); // affiche number
```

```
alert(typeof nothing); // affiche undefined
```

La concaténation

L'opérateur + en plus de faire l'addition, il permet de faire la concaténation.



Conversion de type

Les fonctions de conversion de types chaine/numérique:

Eval() évaluation et conversion numérique d'une chaine

Var a = 2

Eval(a*2) //retourne 4

parseInt() conversion d'une chaine en un nombre entier

parseInt("FF",16) // retourne 255 correspond à la chaine FF dans la base 16

parseFloat() conversion d'une chaine en un nombre réel

parseFloat("61.9") //retourne 61.9



String() transforme un objet en chaîne de caractères

isFinite() permet de tester si la variable passée en paramètre est bien un nombre fini

isFinite(+infinity) // retourne false

isNaN() permet de tester si le paramètre n'est pas un nombre

isNaN("abc") // retourne true



Portée d'une variable

Variable globale: une variable déclarée sans le mot clé **var** et accessible de partout dans le script.

Variable locale: déclarée avec le mot clé **var** dont la portée dépend de l'endroit où elle est déclarée.

Interagir avec l'utilisateur

Prompt() s'utilise comme **alert()** mais a une petite particularité, elle renvoie ce que l'utilisateur a écrit sous forme d'une chaîne de caractères.

Var text = prompt(' tapez quelque chose : ');

// le texte tapé par l'utilisateur se retrouvera stocké dans la variable text.

Convertir une chaine de caractères en un nombre

parseInt()

Convertir un nombre en chaine de caractères

Chaine vide

Text = nbre1 + '' + nbre2



Le plan de la formation

❖ Introduction générale

- Qu'est ce que le JavaScript
- L'historique du JavaScript

❖ Les bases du JavaScript

- Insertion du code JavaScript dans une page html
- Les variables
- **Les opérateurs**
- Les structures de contrôle
- Les fonctions
- Lire/Ecrire

❖ Les événements



Les opérateurs

Les opérateurs arithmétiques

$+$, $-$, $/$, $\%$, $*$

Les opérateurs de comparaison

$<$, $<=$, $>$, $>=$, $==$, $!=$, $===$

Les opérateurs d'affectation complexes:

$+=$, $-=$, $*=$, $/=$, $\%=$

Opérateur ternaire:

Il permet d'affecter une valeur à une variable en fonction d'un résultat à un test.

Variable = test ? Valeur_si_vrai : valeur_si_faux

Les opérateurs logiques

$\&\&$, $||$



Le plan de la formation

❖ Introduction générale

- Qu'est ce que le JavaScript
- L'historique du JavaScript

❖ Les bases du JavaScript

- Insertion du code JavaScript dans une page html
- Les variables
- Les opérateurs
- **Les structures de contrôle**
- Les fonctions
- Lire/Ecrire

❖ Les événements



Les structures de contrôle

- If else
- Switch case
- While
- Do while
- For
- For in



Le plan de la formation

❖ Introduction générale

- Qu'est ce que le JavaScript
- L'historique du JavaScript

❖ Les bases du JavaScript

- Insertion du code JavaScript dans une page html
- Les variables
- Les opérateurs
- Les structures de controle
- **Les fonctions**
- Lire/Ecrire

❖ Les événements



Les fonctions

- Une fonction est un sous-programme qui permet d'effectuer un ensemble d'instructions par simple appel dans le corps du programme principale.
- Une fonction doit être définie avant d'être utilisée
- La déclaration d'une fonction se fait grâce au mot clé **function** selon la syntaxe suivante:

```
function nom_de_la_fonction(arg1,arg2,...)  
{  
  //liste d'instructions  
}
```

- On invoque une fonction avec son nom et sa liste d'arguments entre parenthèses:
nom-de-la-fonction(arg1,arg2,....)
- Une fonction peut éventuellement retourner une valeur à l'aide de l'instruction **return**.

Les fonctions prédéfinies

setTimeout() permet de spécifier un temps après lequel une certaines actions doit s'exécuter

setTimeout(fonction,durée);

// durée en millisecondes

setTimeout("alert('trente secondes sont passées!');",30000);



`clearTimeout()` permet d'arrêter une exécution avec `setTimeout()`

```
Var vTimeout=setTimeout(fonction,durée);
```

```
clearTimeout(vTimeout);
```

`setInterval()`

```
setInterval(fonction, durée);
```

Elle peut être arrêter par `clearTimeout()`



Les fonctions anonymes

Comme leur nom l'indique, ces fonctions sont anonymes car elles ne possèdent pas de nom. Pour la déclarer il faut faire comme pour une fonction classique mais sans indiquer de nom.

```
1 function(arguments) {  
2   //le code de votre fonction anonyme  
3 }
```

Comment exécuter une fonction anonyme?

La solution est simple, on peut assigner notre fonction à une variable.

Exemple:

```
Var sayHello = function() {  
Alert ( ' bonjour ! ');  
};
```

On appelle la fonction par le biais de la variable à laquelle a été assigner:

```
sayHello();
```



Le plan de la formation

❖ Introduction générale

- Qu'est ce que le JavaScript
- L'historique du JavaScript

❖ Les bases du JavaScript

- Insertion du code JavaScript dans une page html
- Les variables
- Les opérateurs
- Les structures de contrôle
- Les fonctions
- Lire/Ecrire

❖ Les événements



Lire/crire

❖ Prompt()

Elle ouvre une boîte de dialogue avec deux boutons OK et Annuler

Elle permet de retourner une information lue

❖ Confirm()

Elle ouvre une boîte de dialogue avec deux boutons OK et Annuler

Elle retourne un booléen

❖ Alert()

Elle permet d'écrire un message dans une fenêtre



```
1  <!DOCTYPE html>
2  <html>
3
4  <head>
5      <title> Interaction!! </title>
6  </head>
7
8  <body>
9
10     <script type="text/javascript">
11         var userName = prompt('Entrez votre prénom :');
12         alert(userName);
13     </script>
14
15 </body>
16 </html>
```



Les évènements



Le document object model (DOM)

Le DOM est une interface de programmation pour les documents XML et HTML.

Il nous permet d'accéder au code XML ou HTML d'un document, modifier , ajouter, déplacer ou même supprimer des éléments HTML.

Grace au DOM la page web est vue comme arbre / hiérarchie d'éléments.

Les événements

- ✓ Onclick : un clic du bouton gauche de la souris sur une cible
- ✓ OnMouseOver : passage du pointeur de la souris sur une cible
- ✓ Onblur : une perte de focus sur une cible
- ✓ Onfocus : une activation d'une cible
- ✓ Onselect : une sélection d'une cible
- ✓ Onchange : une modification du contenu d'une cible
- ✓ Onsubmit : une soumission d'un formulaire
- ✓ Onload : un chargement d'une page
- ✓ Onunload : la fermeture de fenêtre ou le chargement d'une page autre que la courante

Graace au gestionnaire d'événements, on peut associer une action à un événement

onEvenement = "action JavaScript/function()";

`<a href="javascript: alert('Vous avez cliqué !');" >Cliquez-moi !</a>`

`<a href="#" onclick="alert('Vous avez cliqué !'); return false;" >Cliquez-moi !</a>`

Sans DOM

```
1  <!DOCTYPE html>
2  <html>
3
4  <head>
5      <title> Interaction!! </title>
6  </head>
7
8  <body>
9      <span onclick="alert('Hello !');">Cliquez-moi !</span>
10
11 </body>
12 </html>
```

DOM-0

```
1  <!DOCTYPE html>
2  <html>
3
4  <head>
5      <title> Interaction!! </title>
6  </head>
7
8  <body>
9
10     <span id="click">Cliquez-moi !</span>
11
12     <script>
13
14         var element = document.getElementById('click');
15
16         element.onclick = function() {
17             alert("Vous m'avez cliqué !");
18         };
19
20     </script>
21
22 </body>
23 </html>
```



DOM-2

```
1  <!DOCTYPE html>
2  <html>
3
4  <head>
5      <title> Interaction!! </title>
6  </head>
7
8  <body>
9
10     <span id="Toclick">Cliquez-moi !</span>
11
12     <script>
13
14         var element = document.getElementById('Toclick');
15
16         element.addEventListener('click', function() {
17             alert("Vous m'avez cliqué !");
18         }, false);
19
20     </script>
21
22 </body>
23 </html>
```



La méthode sans DOM ne peut pas y utiliser l'objet Event pour récupérer des informations sur l'évènement déclenché.

Par contre le DOM-0 peut l'y utiliser or il a deux problèmes majeurs:

- Il est vieux
- Il ne permet pas de créer plusieurs fois le même évènement

Pour le DOM-2 il permet de :

- créer des événements et même les supprimer
`addEventListener()`
`removeEventListener()`
- créer plusieurs fois le même événement
- d'y utiliser l'objet Event pour récupérer des informations sur l'évènement déclenché.

➔ Il se base sur le principe de capture et bouillonnement (true/false) pour choisir quel événement se déclenche en premier.

